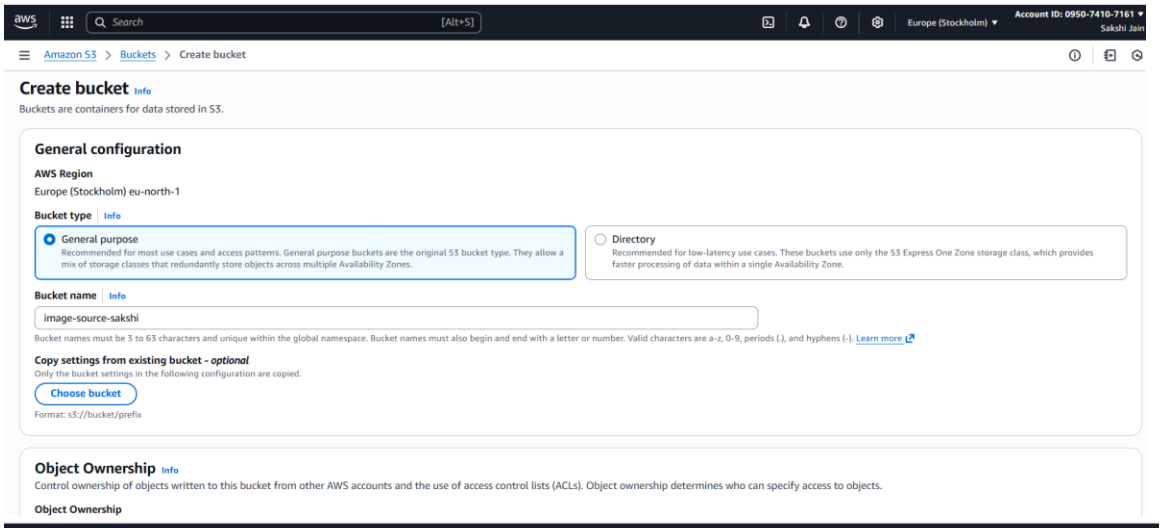
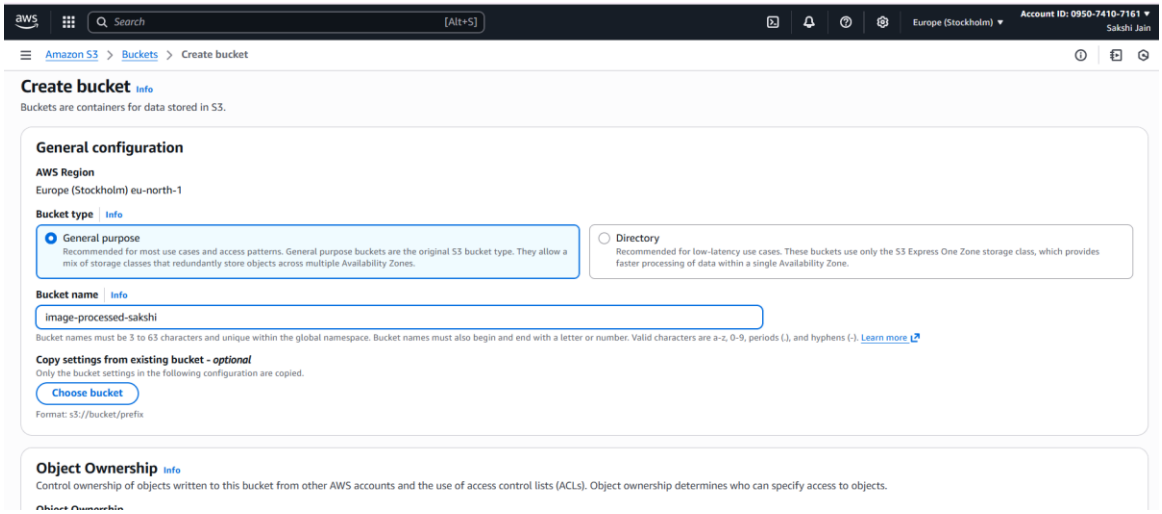
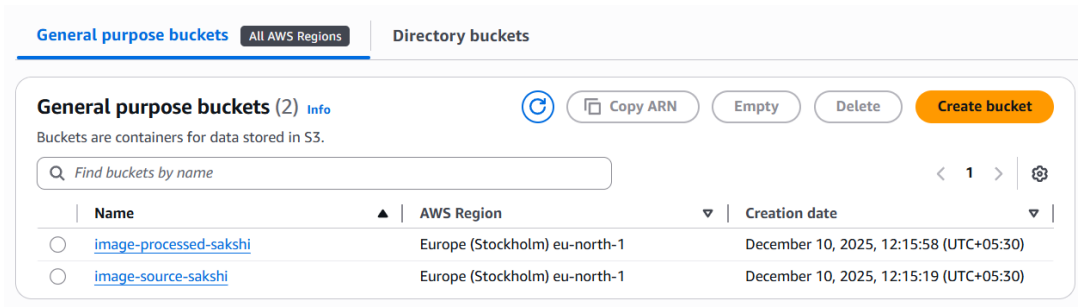
Image Processing and Automation Pipeline Using Lambda & S3 Events

1. **Create S3 Buckets-** Create two buckets







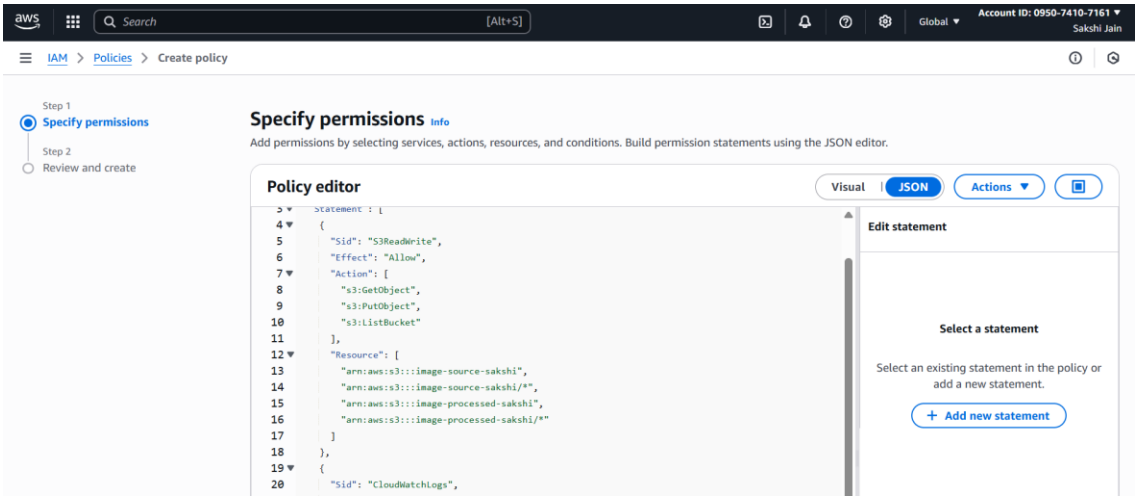
2. **Create IAM Role for Lambda**

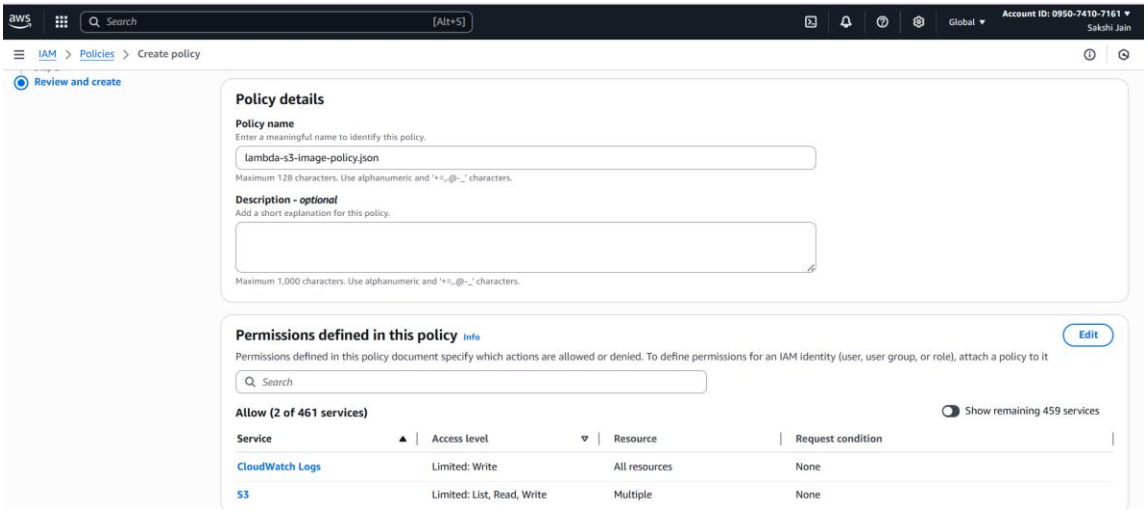
   Lambda requires permissions to read from source bucket, write to destination bucket, and write CloudWatch logs.

   Create IAM policy

```
{
 "Version": "2012-10-17",
```

```
 "Statement": [
  {
   "Sid": "S3ReadWrite",
   "Effect": "Allow",
   "Action": [
    "s3:GetObject",
    "s3:PutObject",
    "s3:ListBucket"
   ],
   "Resource": [
    "arn:aws:s3:::image-source-sakshi",
    "arn:aws:s3:::image-source-sakshi/*",
    "arn:aws:s3:::image-processed-sakshi",
    "arn:aws:s3:::image-processed-sakshi/*"
   ]
  },
  {
   "Sid": "CloudWatchLogs",
   "Effect": "Allow",
   "Action": [
    "logs:CreateLogGroup",
    "logs:CreateLogStream",
    "logs:PutLogEvents"
   ],
   "Resource": "*"
  }
 ]
}
```



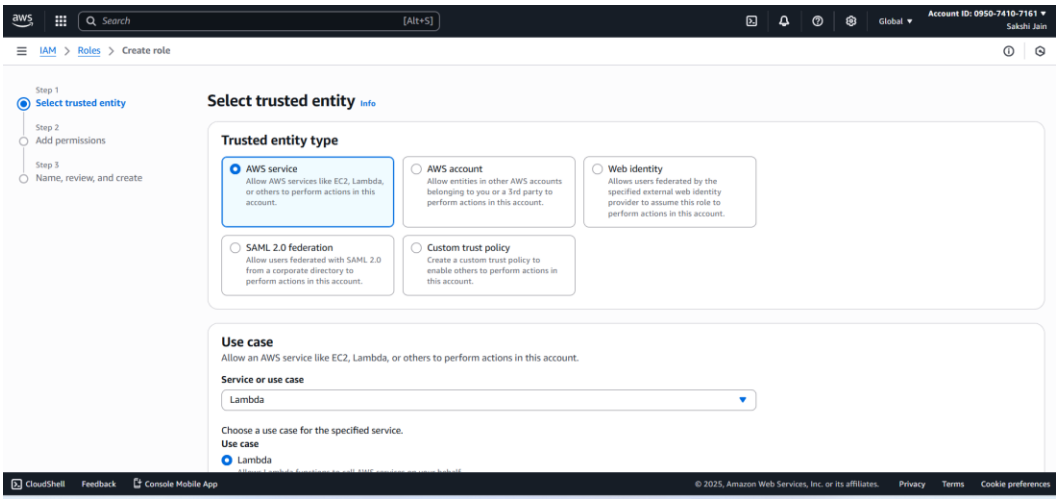


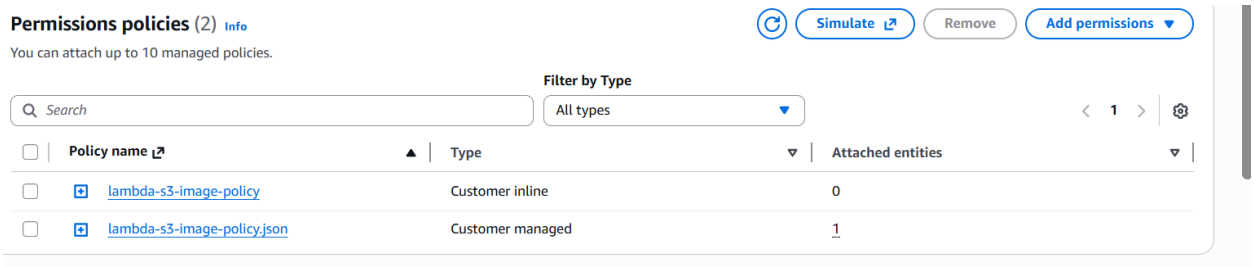
IAM → Role (your Lambda role) → Permissions → Edit policy → JSON

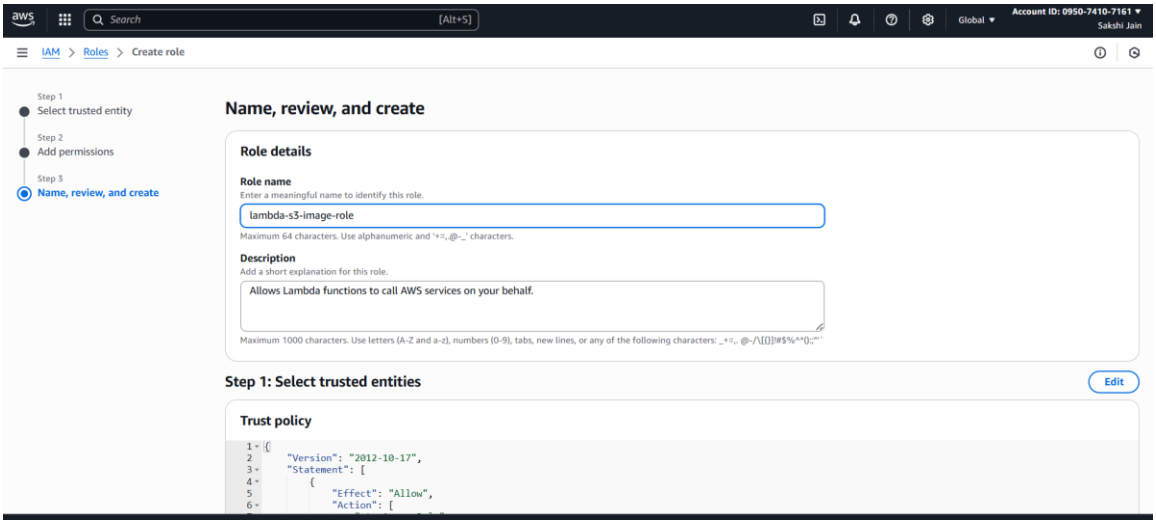
```
{
 "Version": "2012-10-17",
 "Statement": [
   {
    "Effect": "Allow",
    "Action": [
      "logs:CreateLogGroup",
      "logs:CreateLogStream",
      "logs:PutLogEvents"
```

```
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::image-source-sakshi",
        "arn:aws:s3:::image-source-sakshi/*",
        "arn:aws:s3:::image-processed-sakshi",
        "arn:aws:s3:::image-processed-sakshi/*"
      ]
    }
  ]
}
```

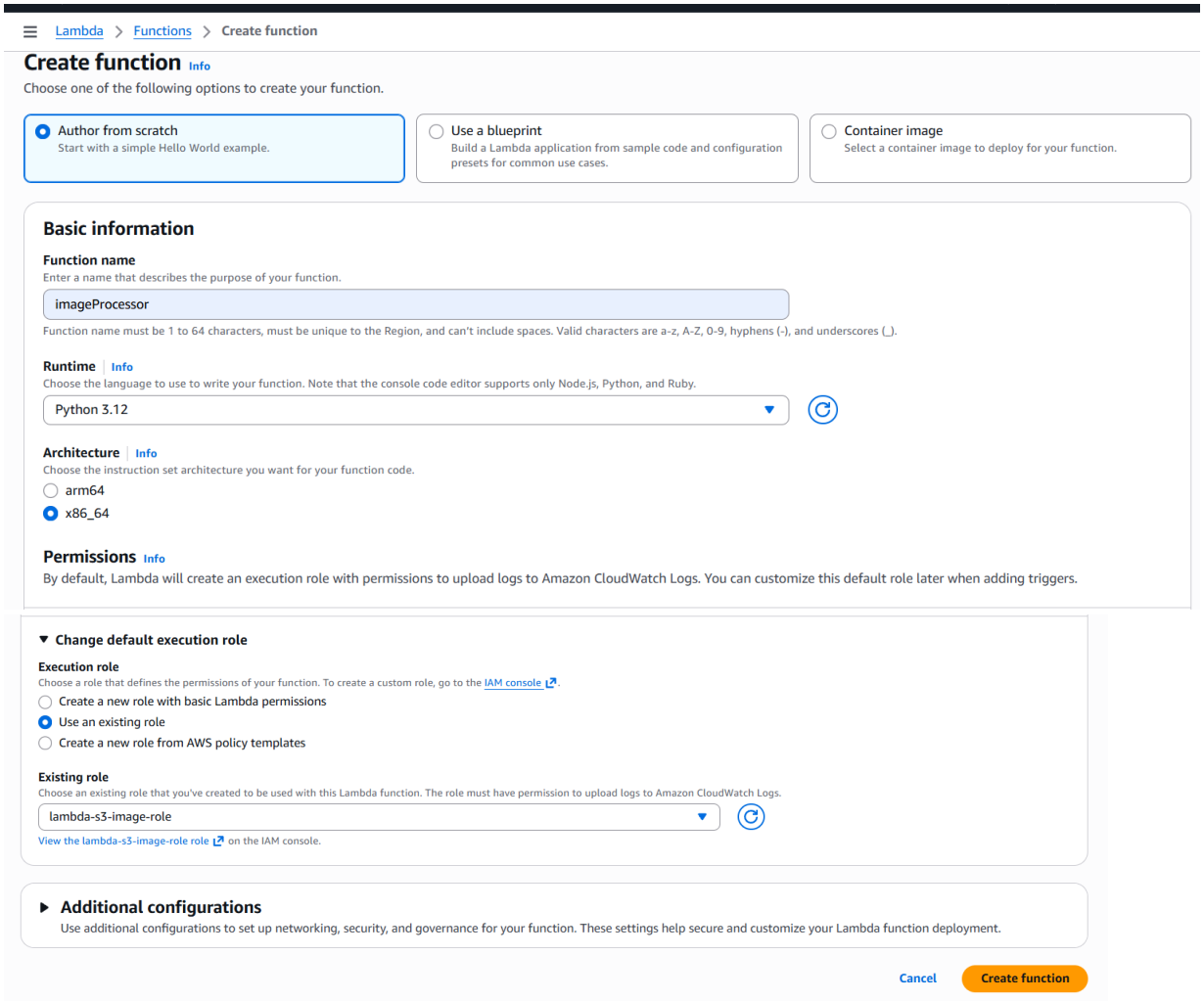
Create role and attach policy





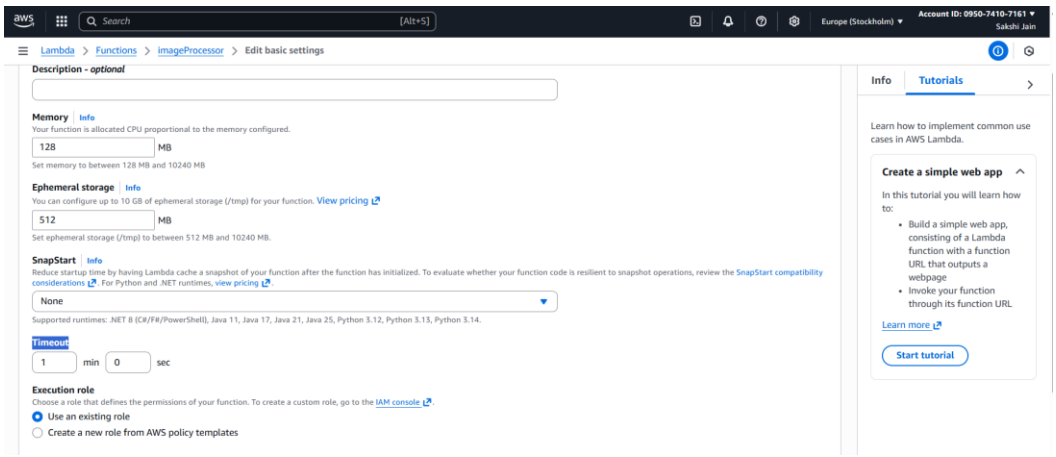


3. Lambda Function Code (Python)



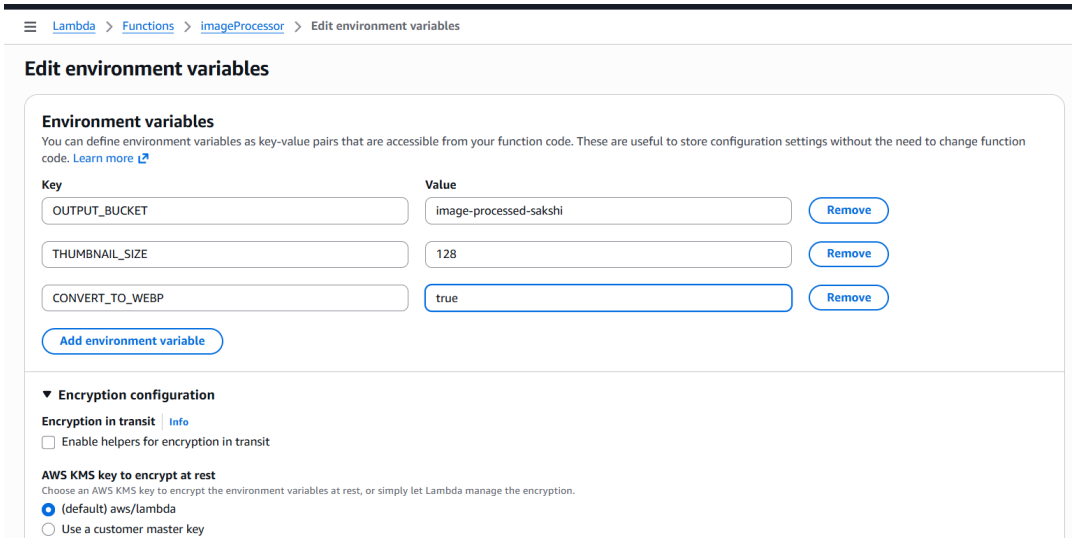
Lambda → Functions → imageProcessor

Go to the **Configuration** tab.



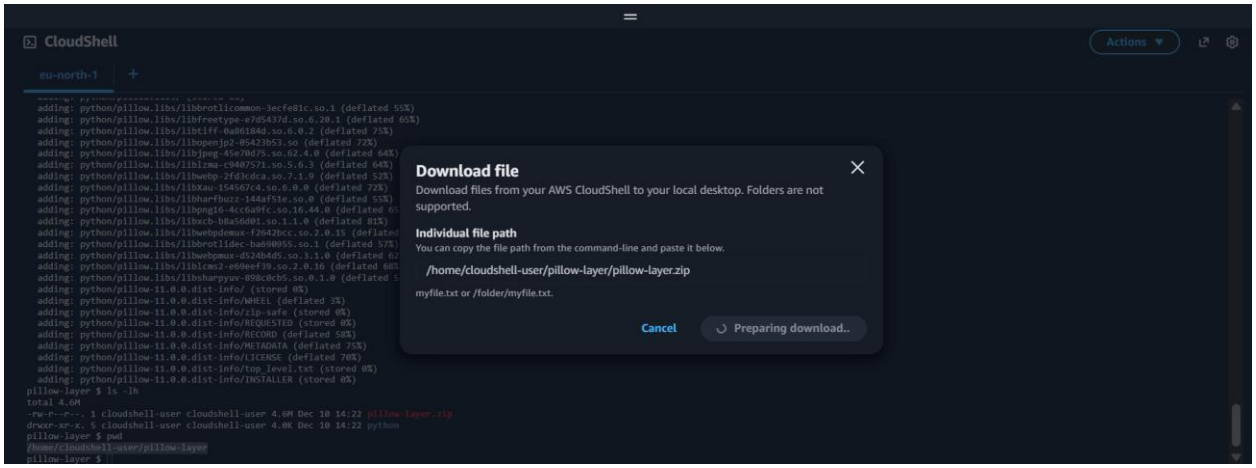
Still on the Configuration tab.

Click Environment variables (left side) → Edit.

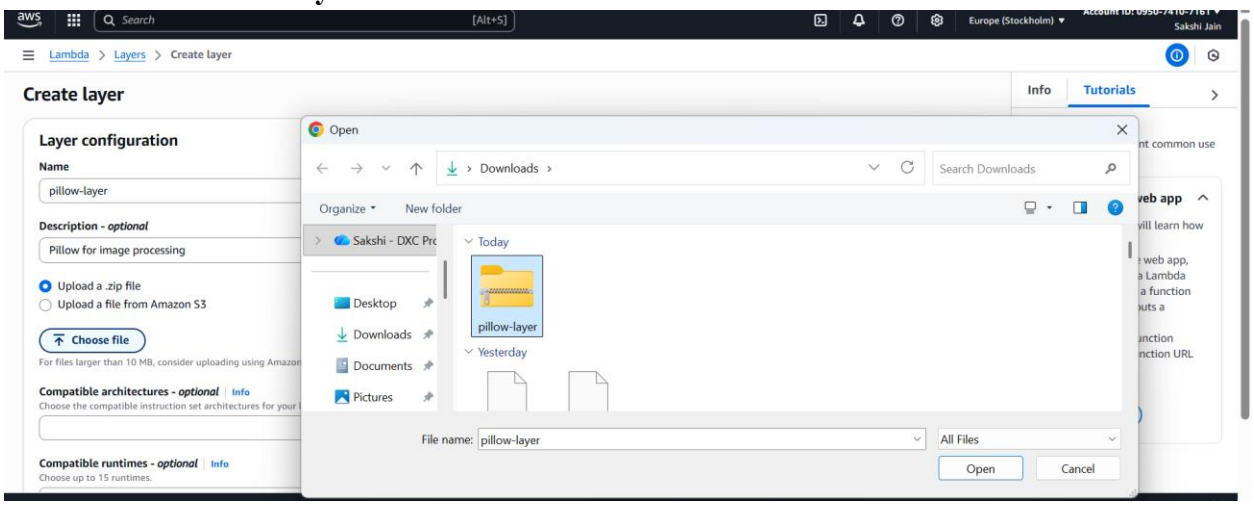


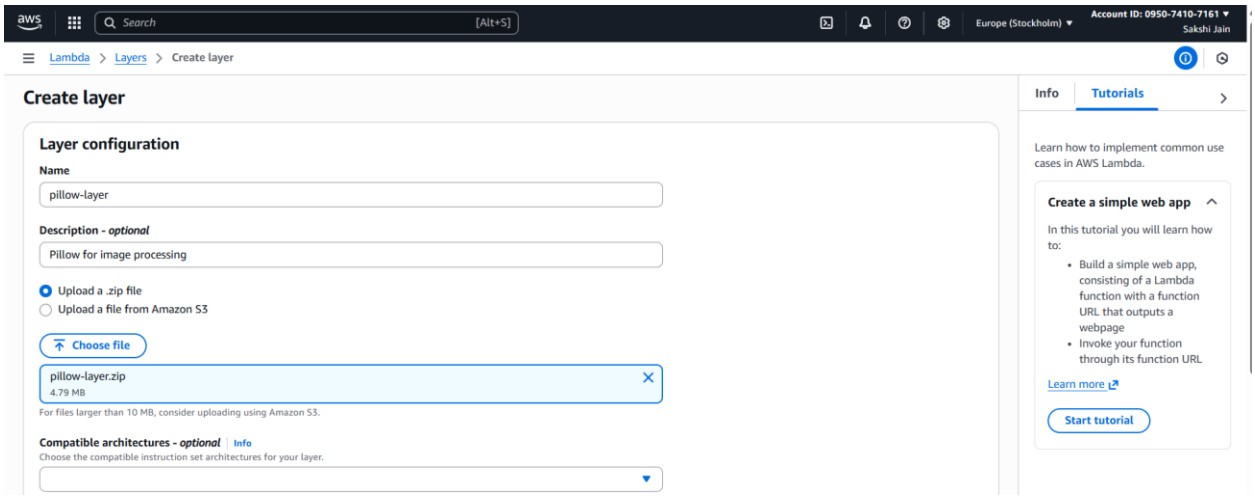
Create your own Pillow Lambda Layer (no Docker)
Open CloudShell
Build the layer contents

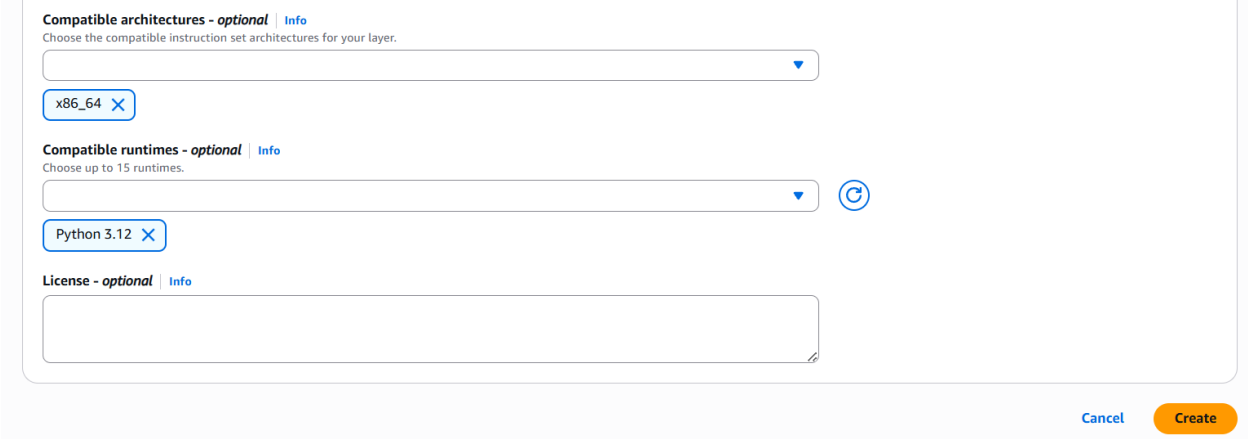
```
# 1. Create a working folder
mkdir pillow-layer
cd pillow-layer

# 2. Create the "python" folder required by Lambda layers
mkdir python

# 3. Install Pillow into that folder
pip3 install "Pillow==11.0.0" -t python/

# 4. Zip it
zip -r pillow-layer.zip python
```



4. **Create the Lambda Layer**

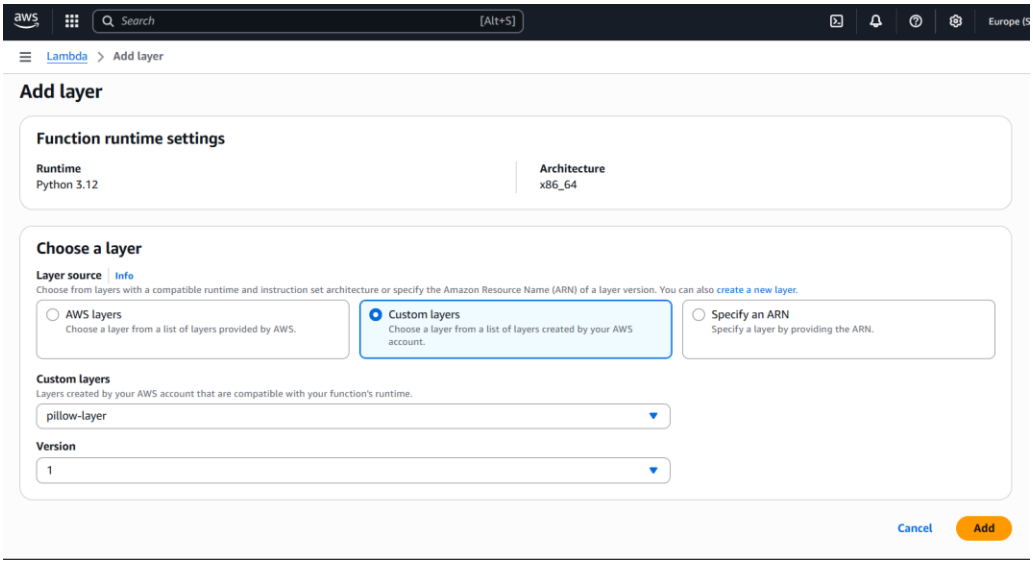






ARN: arn:aws:lambda:eu-north-1:095074107161:layer:pillow-layer:1

Attach the Pillow layer to your Lambda

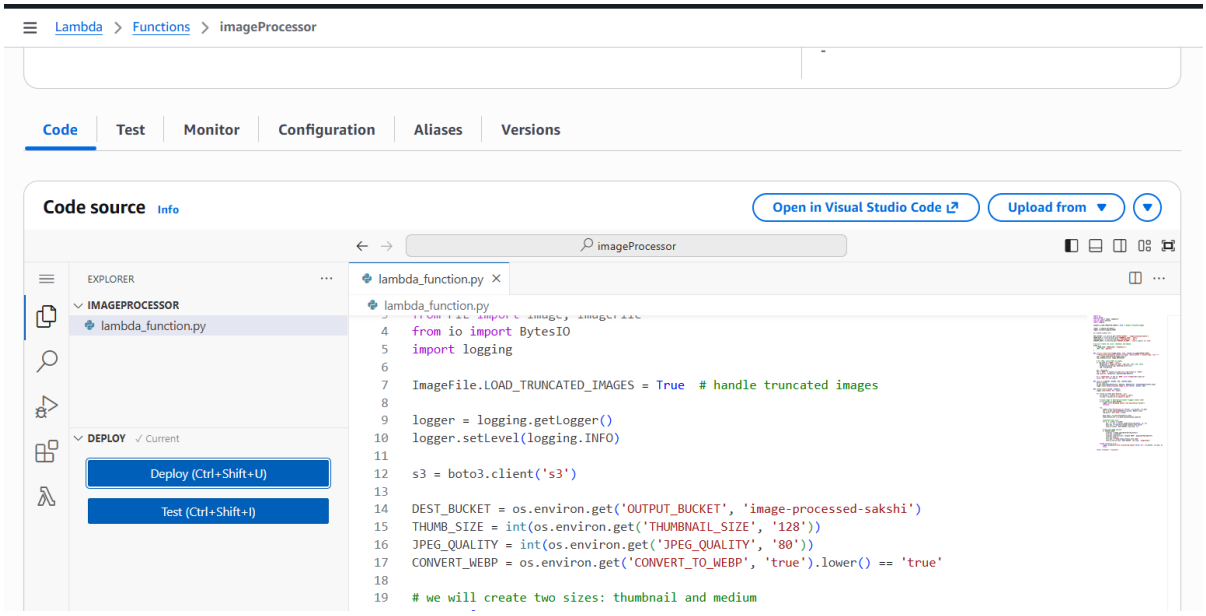


5. **Lambda Code**

In the Code tab of lambda_function

```
import boto3
from PIL import Image
import os
from urllib.parse import unquote_plus

s3 = boto3.client('s3')

def lambda_handler(event, context):
    try:
        # Extract bucket + object key
        record = event['Records'][0]
        source_bucket = record['s3']['bucket']['name']
```

```
        object_key = unquote_plus(record['s3']['object']['key'])  # IMPORTANT FIX

        print("Processing:", source_bucket, object_key)

        # Download file from S3
        tmp_path = f"/tmp/{os.path.basename(object_key)}"
        s3.download_file(source_bucket, object_key, tmp_path)

        # Open and resize image
        img = Image.open(tmp_path)
        img = img.resize((300, 300))

        # Output path
        output_bucket = "image-processed-sakshi"
        output_key = f"processed-{os.path.basename(object_key)}"
        output_path = f"/tmp/{output_key}"

        img.save(output_path)

        # Upload processed image
        s3.upload_file(output_path, output_bucket, output_key)

        print("Upload successful:", output_key)

        return {"status": "success", "file": output_key}

    except Exception as e:
        print("Error:", str(e))
        raise e
```

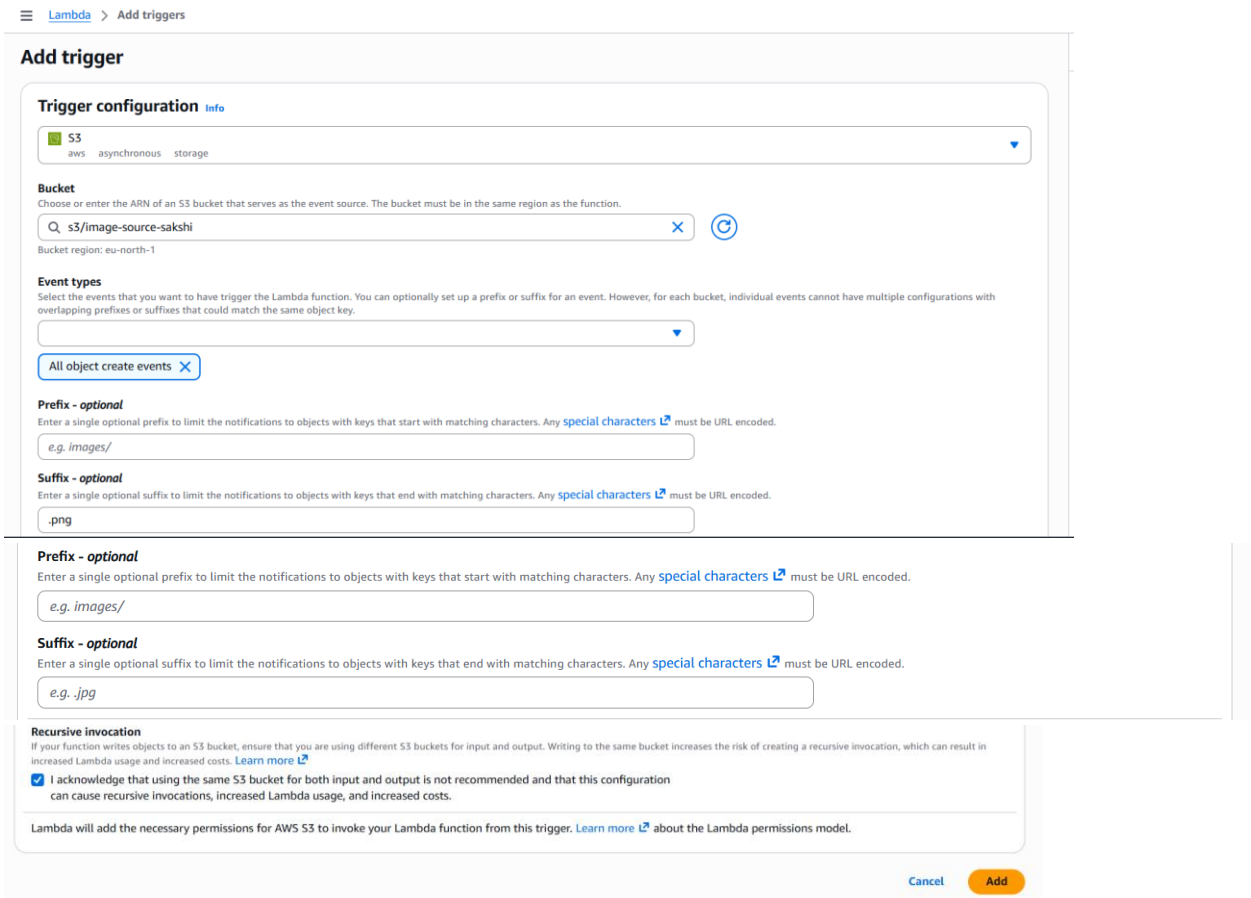


Click **Deploy**.

6. **S3 Trigger Setup (image-source-sakshi → Lambda)**

   We want: **every upload to image-source-sakshi triggers the Lambda**.

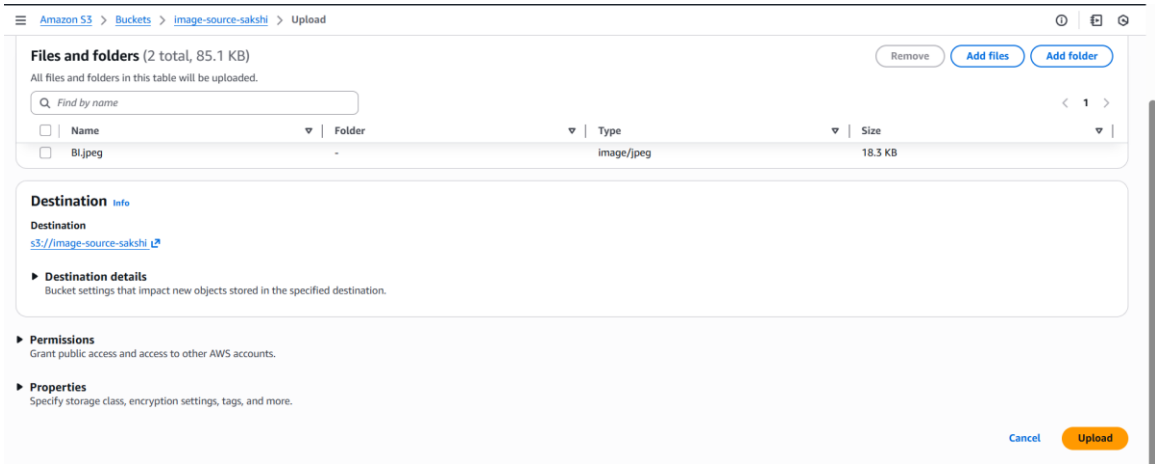
   Lambda → **Configuration** → **Triggers** → **Add trigger**.



7. Test the Pipeline End-to-End

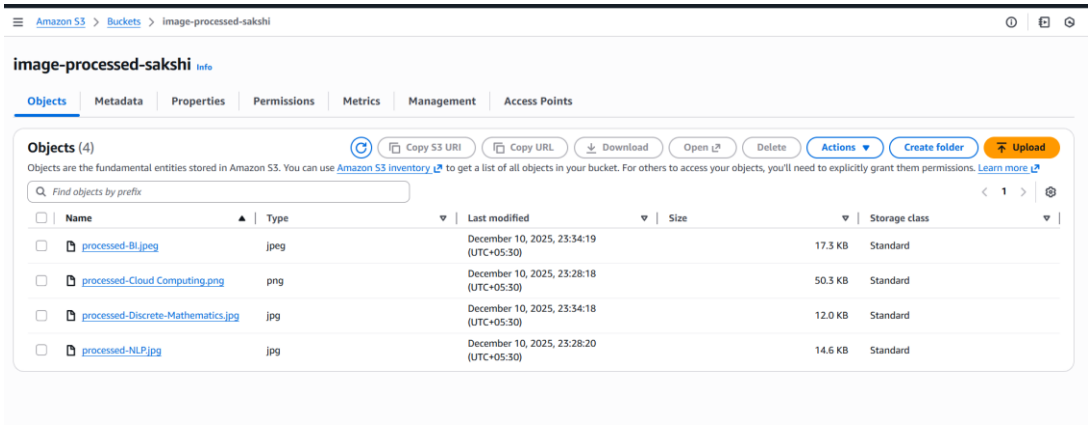
Upload a test image

image-source-sakshi:



Check processed bucket

Look at image-processed-sakshi.



Check logs